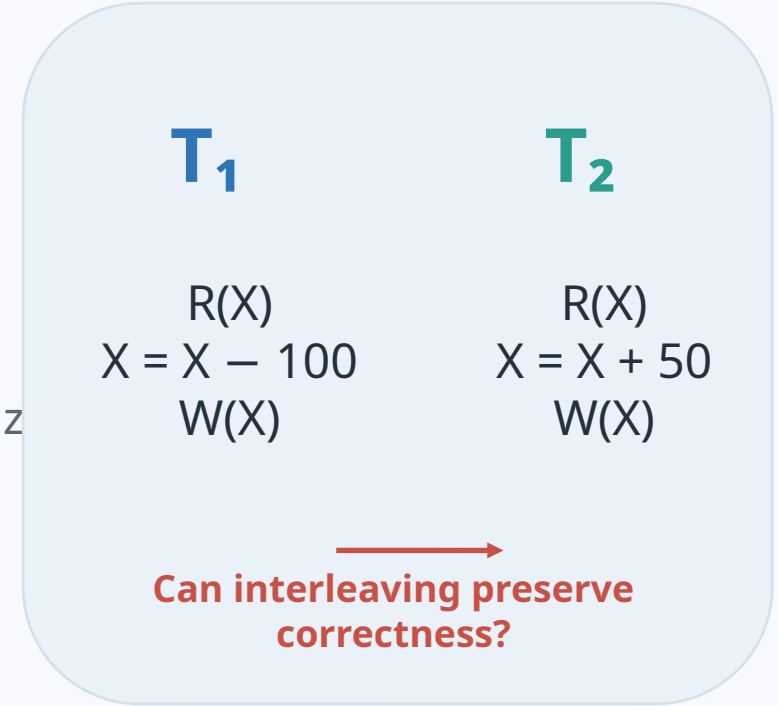
DATABASE ENGINEERING

Unit V: Transaction Management and Serializability

Concepts • Schedules • Concurrency Problems • Conflict & View Serializ



Up to Serializability

Classroom-ready notes with solved examples and practice problems

Learning Outcomes

By the end of this unit, students should be able to:

1. Explain Transactions

Define a transaction, identify transaction states, and explain ACID properties.

2. Analyze Schedules

Differentiate serial and non-serial schedules and identify concurrency anomalies.

3. Test Serializability

Use conflicting operations and precedence graphs to test conflict serializability.

4. Check View Equivalence

Understand read-from and final-write conditions used in view serializability.

5. Solve Exam Problems

Construct schedules, draw serialization graphs, and justify answers step by step.

What is a Transaction?

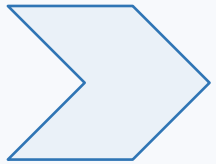
A transaction is a logical unit of database work.

A transaction contains one or more database operations that together perform a meaningful task.

Example: Bank Transfer

Transfer ₹1,000 from account A to account B.

1. Read(A)
2. A = A − 1000
3. Write(A)
4. Read(B)
5. B = B + 1000
6. Write(B)
7. COMMIT



Goal

Either the entire transfer should succeed, or none of it should be reflected in the database.

This all-or-nothing behavior is the foundation of transaction processing.

BEGIN **READ / WRITE** **COMMIT / ROLLBACK**

Basic Transaction Operations

Operation	Meaning	Example
Read(X)	Copy database item X into a local transaction variable	$R_1(X)$
Write(X)	Copy the local value back to database item X	$W_1(X)$
Commit	Make transaction changes permanent	C_1
Abort / Rollback	Cancel transaction and restore previous consistent state	A_1

Notation Example

$$R_1(A) \rightarrow W_1(A) \rightarrow R_1(B) \rightarrow W_1(B) \rightarrow C_1$$

Subscript identifies the transaction.

ACID Properties

The four properties that make transactions reliable.

A • Atomicity

All operations occur completely or none occur.

Failure → rollback.

C • Consistency

A transaction takes the database from one valid state to another valid state.

I • Isolation

Concurrent transactions should behave as if executed independently.

D • Durability

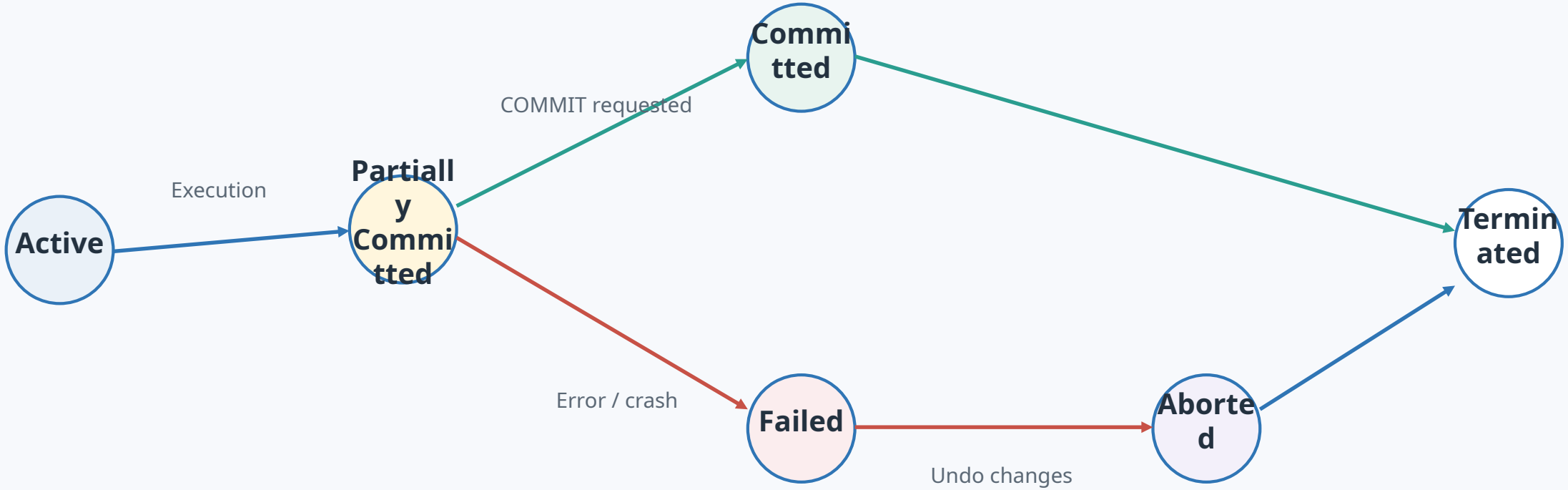
After COMMIT, changes survive failures such as power loss or crash.

Memory Rule

Atomicity = All or nothing • Consistency = Rules preserved • Isolation = Safe concurrency • Durability = Permanent after commit

Transaction States

Typical life cycle of a transaction.



An aborted transaction may be restarted or permanently terminated depending on the cause of failure.

Why Execute Transactions Concurrently?

Better CPU / I/O Utilization

While one transaction waits for disk or network I/O, another can use the CPU.

Higher Throughput

More transactions can be completed per unit time.

Better Response Time

Short transactions do not always need to wait for long transactions to finish.

But concurrency creates a new question:

Is the interleaved execution equivalent to a correct serial execution?

What is a Schedule?

A schedule specifies the order in which operations of multiple transactions are executed while preserving the internal order of each transaction.

Transaction T_1

$R_1(A)$
$W_1(A)$
$R_1(B)$
$W_1(B)$

Transaction T_2

$R_2(A)$
$W_2(A)$
$R_2(C)$
$W_2(C)$

Possible schedule: $R_1(A), W_1(A), R_2(A), W_2(A), R_1(B), W_1(B), R_2(C), W_2(C)$

Serial Schedule vs Non-Serial Schedule

Serial Schedule

One transaction completes before the next starts.

Example:
$R_1(X)$, $W_1(X)$, $R_1(Y)$, $W_1(Y)$, C_1,
$R_2(X)$, $W_2(X)$, C_2

✓ Easy to reason about
✓ Correct if each transaction is correct
✗ Lower concurrency

Non-Serial Schedule

Operations from transactions are interleaved.

Example:
$R_1(X)$, $R_2(X)$, $W_1(X)$, $W_2(X)$, ...

✓ Better concurrency
✓ Better resource use
✗ May cause anomalies
✗ Must be checked for serializability

Important: Non-serial does not automatically mean incorrect. A non-serial schedule can still be serializable.

Concurrency Problem 1: Lost Update

Two transactions read the same old value and one update overwrites the other.

Step	T_1	T_2	X
1	$R_1(X=100)$		100
2		$R_2(X=100)$	100
3	X = X + 20		100
4		X = X − 10	100
5	$W_1(X=120)$		120
6		$W_2(X=90)$	90

Expected after both updates: 110 • Final database value: 90 → T_1 update is lost.

Concurrency Problem 2: Dirty Read

A transaction reads data written by another transaction before that transaction commits.

Step	T_1	T_2	Comment
1	$W_1(X=500)$		T_1 writes new value
2		$R_2(X=500)$	T_2 reads uncommitted value
3	$ABORT_1$		T_1 is rolled back
4		Uses X=500	T_2 used a value that never became valid

Also called: Uncommitted Dependency / Temporary Update Problem

Other Concurrency Anomalies

Unrepeatable Read

T_1 reads X. T_2 updates and commits X. T_1 reads X again and obtains a different value.

$R_1(X=100)$
$W_2(X=120)$, C_2
$R_1(X=120)$

Incorrect Summary

One transaction computes an aggregate while another transaction updates some of the same rows. The aggregate may combine old and new values.

Phantom Read

T_1 executes a condition-based query. T_2 inserts/deletes rows matching the condition. T_1 repeats the query and sees a different set of rows.

These anomalies motivate concurrency-control protocols and the concept of serializability.

Serializability

The correctness criterion for concurrent schedules.

A schedule is serializable if its effect is equivalent to some serial schedule of the same transactions.

Serial Order 1

$T_1 \rightarrow T_2$

All operations of T_1 execute first, then all operations of T_2.

Serial Order 2

$T_2 \rightarrow T_1$

All operations of T_2 execute first, then all operations of T_1.

A non-serial schedule is acceptable when it behaves like at least one valid serial order.

When Do Two Operations Conflict?

Two operations conflict when all three conditions hold:

- They belong to different transactions.
- They access the same data item.
- At least one of them is a WRITE operation.

Operation Pair	Conflict?
$R_1(X)$, $R_2(X)$	No
$R_1(X)$, $W_2(X)$	Yes
$W_1(X)$, $R_2(X)$	Yes
$W_1(X)$, $W_2(X)$	Yes
$W_1(X)$, $R_2(Y)$	No

Conflicting operations cannot be swapped without potentially changing the result.

Conflict Equivalence

Two schedules are conflict equivalent if they contain the same operations and preserve the order of every pair of conflicting operations.

Schedule S_1

$R_1(X)$, $W_1(X)$, $R_2(X)$, $W_2(X)$

Conflicting order:
$W_1(X)$ before $R_2(X)$
$W_1(X)$ before $W_2(X)$

Serial Schedule $T_1 \rightarrow T_2$

$R_1(X)$, $W_1(X)$, $R_2(X)$, $W_2(X)$

Same relative order of all conflicting operations.

Therefore S_1 is conflict equivalent to the serial order $T_1 \rightarrow T_2$.

Conflict Serializability

A schedule is conflict serializable if it is conflict equivalent to some serial schedule.

Method 1: Swapping

Repeatedly swap adjacent non-conflicting operations.

If the schedule can be transformed into a serial schedule, it is conflict serializable.

Method 2: Graph

Construct a precedence or serialization graph.

No cycle → Conflict serializable
Cycle → Not conflict serializable

Preferred in Exams

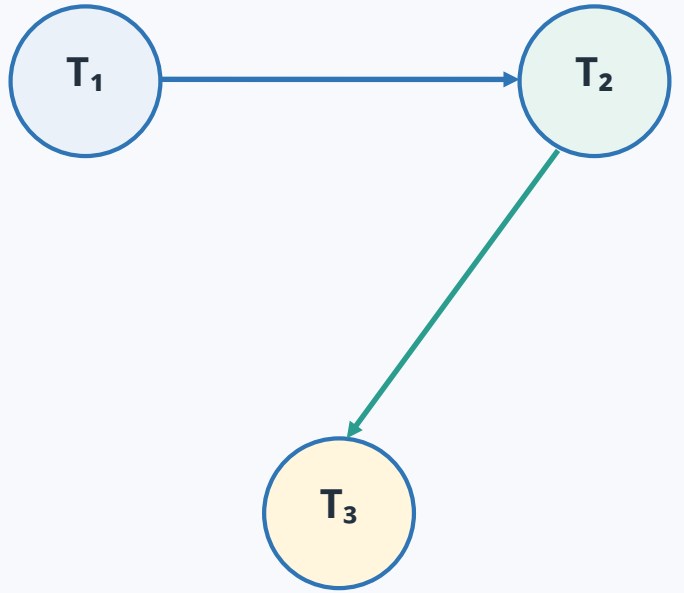
Use the precedence graph because it is systematic and works well for schedules involving many transactions.

Key Theorem: A schedule is conflict serializable if and only if its precedence graph is acyclic.

Precedence Graph: Construction Rules

Create a directed graph G = (V, E).

- One node for each transaction.
- Add edge $T_i \rightarrow T_j$ when an operation of T_i occurs before and conflicts with an operation of T_j on the same item.
- After adding all edges, inspect the graph for cycles.



Acyclic graph

Topological order gives the equivalent serial order.

Solved Example 1: Conflict Serializable

Schedule S: $R_1(X), W_1(X), R_2(X), W_2(X), R_2(Y), W_2(Y)$

Step 1: Find Conflicts

On X:
$W_1(X)$ before $R_2(X)$ → $T_1 \rightarrow T_2$
$W_1(X)$ before $W_2(X)$ → $T_1 \rightarrow T_2$
$R_1(X)$ before $W_2(X)$ → $T_1 \rightarrow T_2$

No reverse edge exists.



No cycle

Equivalent Serial Order: $T_1 \rightarrow T_2$

Conclusion: S is conflict serializable.

Solved Example 2: Not Conflict Serializable

Schedule S: $R_1(X), R_2(X), W_1(X), W_2(X)$

Conflicting Pairs

$R_2(X)$ before $W_1(X)$ → $T_2 \rightarrow T_1$

$R_1(X)$ before $W_2(X)$ → $T_1 \rightarrow T_2$

$W_1(X)$ before $W_2(X)$ → $T_1 \rightarrow T_2$



Cycle: T_1 ↔ T_2

Conclusion: The schedule is NOT conflict serializable.

Solved Example 3: Three Transactions

S: $R_1(A), W_1(A), R_2(A), W_2(B), R_3(B), W_3(C), R_2(C), W_2(C)$

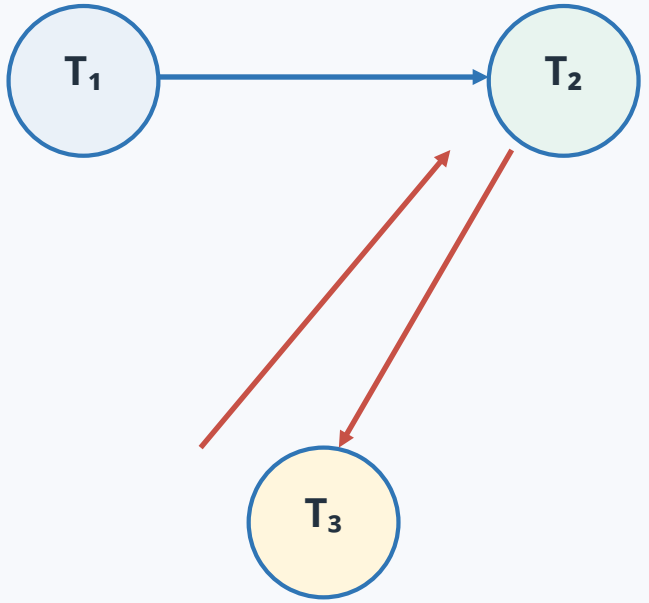
Edges

A: $W_1(A)$ before $R_2(A)$ → $T_1 \rightarrow T_2$

B: $W_2(B)$ before $R_3(B)$ → $T_2 \rightarrow T_3$

C: $W_3(C)$ before $R_2(C)$, $W_2(C)$ → $T_3 \rightarrow T_2$

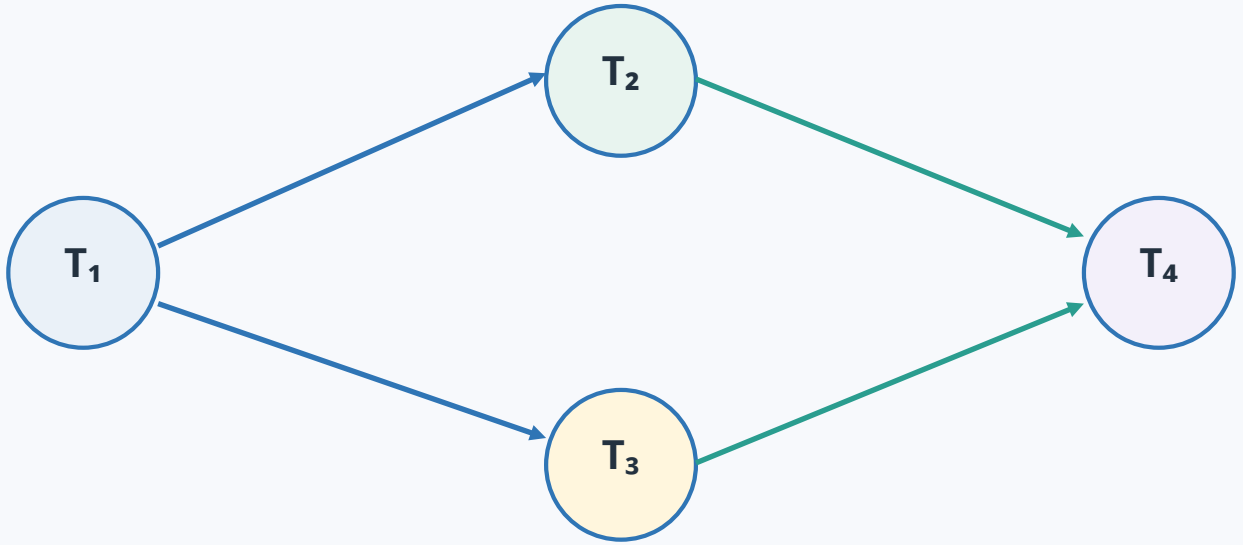
So T_2 ↔ T_3 forms a cycle.



Cycle exists → Not conflict serializable

Topological Order and Equivalent Serial Schedule

When the precedence graph is acyclic, any topological ordering of its nodes gives an equivalent serial order.



Possible serial orders:

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4$
$T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4$

Both are valid because T_2 and T_3 have no ordering edge between them.

View Serializability

A broader notion of serializability.

Two schedules are view equivalent when they preserve the same read-from relationships and the same final writer for each data item.

Condition 1: Initial Read

If T_i reads the initial value of X in one schedule, it must read the initial value of X in the other schedule.

Condition 2: Read-From

If T_i reads X written by T_j in one schedule, the same must hold in the other schedule.

Condition 3: Final Write

The transaction performing the final write on X must be the same in both schedules.

Every conflict-serializable schedule is view serializable, but the converse is not always true.

Blind Write and View Serializability

A blind write occurs when a transaction writes a data item without first reading it.

Example Schedule

$W_1(X)$, $W_2(X)$, $W_1(X)$

T_1 performs the final write on X.
No transaction reads X.

Key Point

Some schedules with blind writes may be view serializable even though their precedence graph contains a cycle, so they are not conflict serializable.

In most exam problems, conflict serializability is tested first because graph checking is simpler.

Conflict vs View Serializability

Aspect	Conflict Serializability	View Serializability
Basis	Order of conflicting operations	Read-from relationships and final writes
Testing	Precedence graph	More complex to test
Blind writes	May reject some valid schedules	Can accept certain blind-write schedules
Relationship	Subset of view-serializable schedules	Superset of conflict-serializable schedules
Exam technique	Draw graph and check cycle	Apply 3 view-equivalence conditions

Conflict Serializable ⊂ View Serializable

Exam Problem 1: Determine Conflict Serializability

Given schedule:

$$S = R_1(X), W_1(X), R_2(Y), W_2(Y), R_1(Y), W_1(Y), R_2(X), W_2(X)$$

Your Task

1. Identify every conflicting pair.
2. Add precedence graph edges.
3. Check for cycles.
4. If acyclic, write the equivalent serial order.

Solution Hint

On X, T_1 operates before T_2 → edge $T_1 \rightarrow T_2$.
On Y, T_2 operates before T_1 → edge $T_2 \rightarrow T_1$.

What does this imply?

Answer: Cycle $T_1 \leftrightarrow T_2$ → NOT conflict serializable.

Exam Problem 2: Find the Serial Order

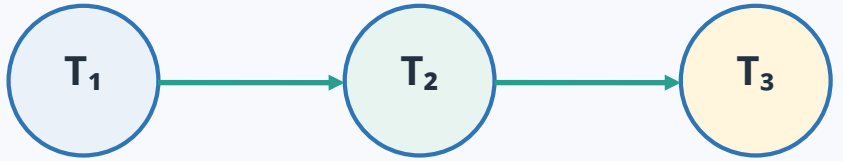
$$S = R_1(A), W_1(A), R_2(A), W_2(B), R_3(B), W_3(C)$$

Conflicts

A: $W_1(A)$ before $R_2(A)$ → T_1 → T_2

B: $W_2(B)$ before $R_3(B)$ → T_2 → T_3

No conflict creates a reverse edge.



Acyclic

Equivalent serial order = $T_1 \rightarrow T_2 \rightarrow T_3$

Therefore, the schedule is conflict serializable.

Practice Problems

Try these before checking the answer key.

- P1. S = $R_1(X)$, $W_1(X)$, $R_2(X)$, $W_2(X)$. Draw the graph and test serializability.
- P2. S = $R_1(X)$, $R_2(Y)$, $W_1(X)$, $W_2(Y)$. Is the schedule conflict serializable?
- P3. S = $R_1(A)$, $W_1(A)$, $R_2(A)$, $W_2(B)$, $R_3(B)$, $W_3(C)$, $R_1(C)$. Find all graph edges.
- P4. Explain why $R_1(X)$ and $R_2(X)$ do not conflict, but $R_1(X)$ and $W_2(X)$ do.
- P5. Give one schedule that demonstrates a lost-update anomaly.
- P6. State the three conditions for view equivalence.
- P7. Can a view-serializable schedule fail to be conflict serializable? Explain the role of blind writes.

Practice Answer Key

P1 to P4

P1: $T_1 \rightarrow T_2$ only, so serializable as $T_1 \rightarrow T_2$.

P2: No conflicts across different items; graph has no edge, so serializable. Either order is possible.

P3: $T_1 \rightarrow T_2$ on A; $T_2 \rightarrow T_3$ on B; $T_3 \rightarrow T_1$ on C. Cycle exists, so not conflict serializable.

P4: Read-read does not change data. Read-write may change the value seen or final result.

P5 to P7

P5: $R_1(X)$, $R_2(X)$, $W_1(X)$, $W_2(X)$ can cause lost update.

P6: Preserve initial reads, read-from relationships, and final writer.

P7: Yes. View serializability is broader. Blind writes can create schedules that are view serializable but not conflict serializable.

Common Mistakes to Avoid

Mistake 1

Adding an edge for every pair of operations. Add edges only for conflicting operations on the same item.

Mistake 2

Treating read-read as a conflict. $R_i(X)$ and $R_j(X)$ do not conflict.

Mistake 3

Looking only for a direct $T_1 \leftrightarrow T_2$ cycle. Longer cycles such as $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$ also matter.

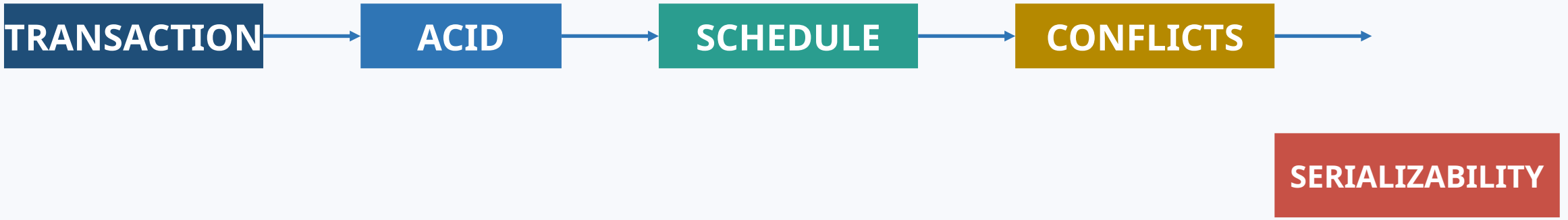
Mistake 4

Writing a serial order without first confirming that the precedence graph is acyclic.

Mistake 5

Confusing conflict serializability with view serializability. They are related but not identical.

Unit Summary: Up to Serializability



- Serial schedules are inherently easy to reason about, but concurrency improves performance.
- A non-serial schedule is correct when it is equivalent to a valid serial execution.
- Conflict serializability is tested using a precedence graph: acyclic means serializable.
- View serializability preserves initial reads, read-from relations, and final writes.
- Conflict serializability is a subset of view serializability.

Next topic after serializability: concurrency-control protocols such as locking and timestamp ordering.